

Communicating with a compatible ETHerNDE Instrument over Virtual COM Port or RS232

Version	Document Changes	Updated by	Date
1.0	Created	Ian Drew	10-4-2015
1.1	Corrected errors in Button Press command values	Ian Drew	19-8-2016
1.2	New command to request specified number of packets	Ian Drew	26-9-2016
1.3	Added the new formats of Conductivity, C-Scan and Non-Realtime.	Ian Drew	13-8-2018
1.4	Added RS232 commands for Victor and added RS232 to the title	Ian Drew	21-5-2019

Contents

Contents	2
Introduction	3
Transmitting data from PC Host	3
Predefined Commands	3
Sending XML Strings.....	4
Special XML Commands	5
Sending Files	5
Sending a request for the Instrument to send a file.....	6
Requesting File Listing.....	6
Receiving data from Instrument	7
Description of Status Byte Identifiers	7
Receiving a File from the Instrument.....	11
RS232 Communications	13
Remote Navigation Commands	13
Requesting data from the Victor2.2D	13
Setting data on the Victor2.2D	14
Using ETherRealtime, RS232	14
Keypad.....	15
Settings parameters.....	15

Introduction

The later advanced ETHERNDE instruments with USB can communicate with a PC using USB. We use the Virtual COM Port protocol from ST Microelectronics (STM). Once the STM driver has been installed (google **STM VCP drivers** or use <http://www.st.com/web/en/catalog/tools/PF257938>), any software running on the PC can communicate with the ETHER Instrument as if it were a standard serial COM port. This document describes the required format of data that is sent and received by the instrument.

Transmitting data from PC Host

There are 3 types of data that can be sent to the Instrument.

1. Predefined Commands.
2. XML Strings.
3. Files (settings or standard files).

All data sent TO the instrument is sent in packets up to 64 bytes long. The start, and only the start of each packet must contain 2 or more **Command bytes** from the **Predefined Commands** table, below. Even when sending XML strings or Files, the first 2 bytes are still **Command Bytes**.

Predefined Commands

All commands are constructed using just a few bytes that are sent in order. The first byte is always 0x01. For the 2nd byte onwards see the table below:

2 nd Byte	3 rd Byte (if applicable)	Description	
0x01		The following data is a file name.	See Sending Files .
0x02		File name has completed, prepare for file reception.	
0x03		The following data is file data.	
0x04		File data transmission has finished.	
0x05		Send the micro SD card contents listing.	See Requesting File Listing
0x06	Number of packets to transmit.	Send only the specified number of packets of USB data (using the current format). 0-255. 0=Resume continuous sending.	Only use this if continuous streaming from the instrument is NOT required.
0x08	File number LSB. 4 th Byte is File MSB.	Delete the Nth file in the micro SD card listing.	
0x09	0=External card, 1=Internal Card (if applicable)	Send the Nth file from the micro SD card listing. 4 th Byte File number LSB. 5 th Byte is File MSB.	
0x0A		Reprocess the XML values stored in the instruments Master Settings. This is required after sending an XML string.	See Sending XML Strings .
0x0B		Reprocess the file MS.xml (master settings file).	See Sending

			XML Settings File.
0x0C		Save existing settings in MS.xml	
0x0D	4 th Byte onwards is ASCII.	4 th Byte onwards is a NULL terminated ASCII string containing the size of a file that is about to be transmitted.	
0x0	0x11	OK/Power key	Send key press
	0x01	Menu/Cancel key	Send key press
	0x02	BALANCE 2 (lower Balance key)	Send key press
	0x03	UP key	Send key press
	0x04	DOWN key	Send key press
	0x05	LEFT key	Send key press
	0x06	RIGHT key	Send key press
	0x07	BLANK 2 (lower Blank key)	Send key press
	0x08	Send Datalog Data (SigmaCheck only)	
	0x08	BALANCE 1 (upper Balance key) (NON-SigmaCheck)	Send key press
	0x09	BLANK 1 (Upper Blank key)	Send key press
	0x0A	FREEZE key	Send key press
	0x0D	Long press on Menu/Cancel key	Send key press
	0x11	Toggle Virtual Comms Debugging (send incrementing numbers rather than data)	
	0x12	Load VeeScan Settings (AeroCheck+ Only)	
	0x13	Long press on BLANK 2	Send key press
	0x15	Long press on BLANK 1	Send key press
	0x17	Long press on OK/Power key	Send key press
	0x19	Turn ON VeeScan Beadseat Compensation (AeroCheck+ Only)	
	0x1A	Turn OFF VeeScan Beadseat Compensation (AeroCheck+ Only)	
	0x3C	This is the ASCII of '<' which will be the first character of an XML string.	See Sending XML Strings .

Sending XML Strings

All settings on the instrument are saved in XML form on the microSD card. If one value of the instrument is required to be modified then this can be done by sending a valid ASCII XML string. The exact command syntax must be known! Values are range checked by the instrument. Looking at either the source code file **XML_handler.c** or an existing settings file will give a good indication as to the format. An example of changing the size of the 2nd Pane using **XML_handler.c** for guidance:

```
XML_PANE_SIZE,
UNSIGNED_8,           //type
0,                    //ptr_to_MS
0,                    //minimum
50,                   //maximum
0,                    //WriteChildXML
ReadPaneValues,       //ReadChildXML
```

```
//Size of the 2nd pane in % of screen
```

The XML tag will be "<PANE_SIZE>". The allowable values between 0 and 50. The value must be an unsigned 8bit integer. So, changing the pane 2 size to 35%:

<PANE_SIZE>35</PANE_SIZE>

The above ASCII must be preceded by 0x01, 0x00, as explained in the table above, therefore the command to send would be (including a NULL termination (0x00)):

0x01, 0x00, 0x3C, 0x50, 0x40....., 0x00

Some XML commands are children of other XML commands, they are slightly more complicated.

Using an existing XML file as a basis and changing the 2nd Frequency to 300kHz, the following commands must be sent. Each command must be sent separately, all with the preceding 0x1,0x0 and terminating 0x0:

<DUAL_FREQUENCY>

<FREQUENCY>300000</FREQUENCY>

</DUAL_FREQUENCY>

Sending the XML command will not cause the instrument to instantly use the modified value. To correctly do this, issue the Reprocess XML Values command, see the above table!

Special XML Commands

There some XML commands that are handled instantly rather than just changing the value of a variable. These are explained below:

Changing the USB Data output format – Send XML command "<USB_OUTPUT>". The number to send after the opening tag can be one of the following

0 - Stop USB Realtime output.

4 - Transmit RAW data.

5 - Transmit Processed data (Ch1, Ch2 & Mix)

7 - Single Channel, processed with user configured counter.

Example, the complete packet required to send to instruct the instrument to transmit RAW data would be the **Command Bytes** to send an XML string (0x1, 0x0) followed by the ASCII:

<USB_OUTPUT>4</USB_OUTPUT> followed by a NULL (0x0) .

Once the instrument has received a **Change USB Data Output** command it first transmits up a complete XML Header to the PC. Without this information that contains all of the settings of the instrument, any realtime data is meaningless. See the description of **XML_Header**

Sending Files

Sending files to the instrument is split in to the following steps:

1. Send the path command followed by where to save the file on the micro SD card ie:

To save the file **picture.bmp** to the folder at **\EtherCheck\Saves** send:

0x1, 0x1, '\','E','t','h','e','r','C','h','e','c','k','\','S','a','v','e','s','\','p','i','c','t','u','r','e','.','b','m','p', 0x0

The packet is a maximum of 64 bytes, so if the path is longer send it in up to 4 separate packets. Each packet must start with the command 0x1, 0x1.

2. Send the command to inform the instrument that the sending of the Path and Filename is complete: 0x1, 0x2,
3. Send the size of the file to follow. This is in ASCII after the command bytes, ie file size of 901bytes: 0x1, 0x13, '9', '0', '1', 0x0
4. Send the file contents in packets. Each packet is 64 bytes, the first 2 of which are always the command bytes 0x01, 0x03. Continue sending this until the whole file is sent.
5. Send the file complete command in a packet: 0x1, 0x4.

Sending a request for the Instrument to send a file.

A request for the Instrument to send a file from the microSD card. The request must contain a file number. This file number can only be obtained from an up to date file listing from the Instrument, see **Requesting File Listing** below. Assuming we want the Instrument to send the 5th file on the microSD card we would send the bytes:

0x1

0x9 **Send File** command

0x0 External micro SD Card.

0x5 Fifth file requested.

To send the 260th file:

1,9,0,4,1

The file will then be sent by the Instrument. See **Host PC Reception of VCP Data** below.

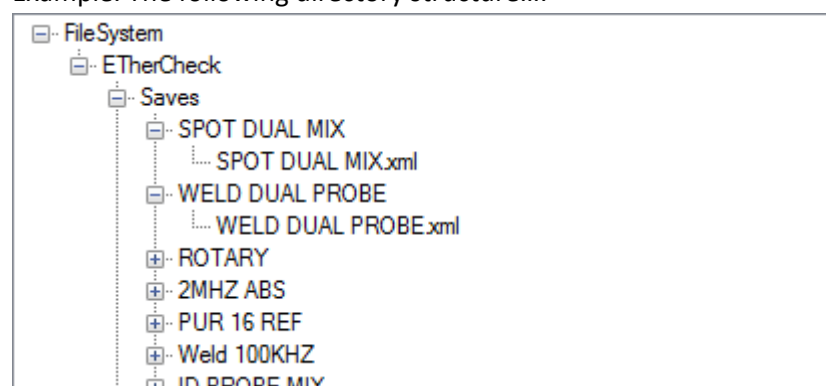
Requesting File Listing

If a file is required from the instrument, the request is made using the position of the file in a list that must first be obtained from the instrument. The request for a file listing is by issuing the command: 0x01, 0x05. The instrument then sends a file to the host PC using the usual routine for sending a file, except this file is named **FS.txt**. See **Receiving a File from the Instrument**.

This file contains a list of all files and directories on the microSD Card but with some clever formatting that allows the layout of the directory structure to be ascertained.

1. Each file or directory is comma separated.
2. If the item is a directory, its name is followed by a '/' and all following items are members of that directory until....
3.a '\ ' is seen, the following item belongs to the level above in the heirachy.

Example. The following directory structure....



Is created from this file....

ETHerCheck/, Saves/, SPOT DUAL MIX/, SPOT DUAL MIX.xml, \WELD DUAL PROBE/, WELD DUAL PROBE.xml, \ROTARY/, ROTARY.xml, \2MHZ ABS/, 2MHZ ABS.xml, \PUR 16 REF/, PUR 16 REF.xml, \Weld 100KHZ/, Weld 100KHZ.xml, \ID PRO.....

Receiving data from Instrument

Data is sent continuously from the instrument once connected (unless data transfer is deliberately cancelled from the PC). All data sent FROM the instrument first contains a Status byte then the inverse of this status byte followed by the data. The size of the expected data is always known in advance so that processing the data in real time is simpler and the data can span different reads from the serial port. File names are the only data that can be less than the specified length as the string can be NULL terminated.

All realtime data, be it raw or post processed, is sent as signed 32bit integers (4 bytes) for the X channel and then for the Y channel. There is no delimiter between each X, Y or channel BUT each packet of values begins with the status byte and then its inverse. Realtime data is sent in 1 of 3 formats:

1. Raw. This sends up raw, unprocessed values of X & Y for both channels.
2. Processed. This sends up X & Y values for 2 channels plus a mix channel.
3. Single Channel processed. This sends up an X & Y for a single channel then another 32 bit integer that has a user configurable purpose.

These are explained below in more detail.

Other data that can be sent from the instrument are Files and XML.

Both begin with the file size being received taking up 10 bytes after 2 bytes for the Status byte and its inverse. Then follows either an **XML_HEADER** or **FILE_DATA** Status byte, its inverse then the data of the file or header. The number of bytes sent is equal to the file size sent up before hand. Finally, if it is a file that has been sent, then the **NULL** terminated name of the file is received after the **FILE_NAME** status byte and its inverse.

All of these commands are explain in more detail below.

Description of Status Byte Identifiers

The status byte is made up of 4bits for an identifier as to what the data packet contains and 4 bits that can be used for various other information, ie Alarm State, Sync Pulse etc. The Most Significant Nibble is the identifier:

FILE_SIZE	= 0x1;
FILE_DATA	= 0x2;
FILE_NAME	= 0x3;
REALTIME_DATA_RAW	= 0x4;
REALTIME_DATA_POSTPROCESS	= 0x5;
XML_HEADER	= 0x6;
SINGLE_CHAN_POST	= 0x7;
NON_REALTIME	= 0x80;
CONDUCTIVITY	= 0x90;
VEE_SCAN_RESPONSE	= 0xA0;
CSCAN_ENC_TRIGGERED	= 0xB0;

The Least Significant Nibble contains the following information:

Rotary Sync Pulse = 0x1;

Alarm State = 0x2;

If a rotary probe is running on the instrument each rotation triggers a sync pulse. This is sent in the data as the LSB. If an alarm is active on the instrument the 2nd bit is set.

File_Size

Following this, there will be 10 bytes. They contain a number that indicates the length of a file that is about to be transmitted. The number is in binary (not ASCII). The number is right aligned within the 10 bytes. The first 5 bytes MUST be 0, this still leaves us with a maximum file size of 5^{256} or a Terrabyte! Example, a file size of 1740bytes:

```
0x10    Status Byte
0xEF    Inverse Status byte
0x0
0x0
0x0
0x0
0x0
0x0
0x0
0x0
0x0
0x6      0x6 = 6. x 256 = 1536
0xCC     0xCC = 204. 204+1536 = 1740.
```

File_Data

Following this status byte will be a number of bytes identified by the File_Size already received. This data is also binary and can be outputted directly to an open file.

File_Name

After a complete file has been received the FILE_NAME status byte is sent. It is a NULL terminated ASCII string that holds the file name of the file just received. Maximum of 256 characters.

Realtime_Data_Raw

Raw data from the instrument will contain 4 values, an XY pair for Channel 1 and an XY pair for Channel 2. Each value is 4 bytes (therefore 16 in total). The exact order is X1, Y1, X2, Y2. An example packet:

```
0      0x40    Status byte
1      0xbf    Inverse status byte
2      0x00    X
3      0x31    X
4      0x97    X
5      0xf8    X      0xf8973100 = -124309248
6      0x00    Y
7      0x01    Y
8      0xcc    Y
9      0x0e    Y      0x0ecc0100 = 248250624
10     0x00    X2
11     0x4d    X2
12     0x75    X2
13     0xfd    X2      0xfd754d00 = -42644224
14     0x00    Y2
```

15	0xaa	Y2	
16	0x66	Y2	
17	0x14	Y2	0x1466aa00 = 342272512

Realtime_Data_Postprocess

As the data has already been processed, there is also the Mix channel that is returned so we have 3 pairs of XY values. PLEASE NOTE the data is sent back in a different order, X1, X2, Xmix, Y1, Y2, Ymix.

0	0x50	Status byte	
1	0xaf	Inverse status byte	
2	0x9c	X1	
3	0xff	X1	
4	0xff	X1	
5	0xff	X1	0xffffffff9c = -100
6	0x7b	X2	
7	0xfa	X2	
8	0xff	X2	
9	0xff	X2	0xffffffffa7b = -1413
10	0x20	Xmix	
11	0x05	Xmix	
12	0x00	Xmix	
13	0x00	Xmix	0x00000520 = 1312
14	0x58	Y1	
15	0xff	Y1	
16	0xff	Y1	
17	0xff	Y1	0xffffffff58 = -168
18	0xc8	Y2	
19	0xf7	Y2	
20	0xff	Y2	
21	0xff	Y2	0xffffffff7c8 = -2104
22	0x8f	Ymix	
23	0x07	Ymix	
24	0x00	Ymix	
25	0x00	Ymix	0x0000078f = 1935

Single_Chan_Post

Only Channel 1s data is sent (1 XY pair) after processing, followed by a pair of user configurable values.

0	0x70	Status byte	
1	0x8f	Inverse Status byte	
2	0x6f	X1	
3	0xff	X2	
4	0xff	X3	
5	0xff	X4	0xffffffff6f = -145
6	0xe9	Y1	
7	0xff	Y2	
8	0xff	Y3	
9	0xff	Y4	0xffffffffe9 = -23
10	0x40	X1	
11	0x0	X2	
12	0x0	X3	
13	0x0	X4	0x40 = 64
14	0x5	Y1	

15	0x0	Y2	
16	0x0	Y3	
17	0x0	Y4	0x5 = 5

Non_Realtime

50 bytes are always returned, 2 for the status bytes and then 12 lots of 48. These are not all currently used but allow expansion in the future. The current use for the AeroCheck+ range is:

X1, Y1, X2, Y2, Theta, Radius, X%, Y%

0	0x80	Status byte	
1	0x7f	Inverse Status byte	
2	0x59	X1	
3	0x00	X2	
4	0x00	X3	
5	0x00	X4	0x59, X = 59
6	0x67	Y1	
7	0xff	Y2	
8	0xff	Y3	
9	0xff	Y4	0xffffffff67, Y = -153
10	0x00	X1	
11	0x00	X2	
12	0x00	X3	
13	0x00	X4	0x0, x2 = 0
14	0x00	Y1	
15	0x00	Y2	
16	0x00	Y3	
17	0x00	Y4	0x0, y2 = 0
18	0x78		
19	0x00	0x78	Theta = 120
20	0x00	Not Used	
21	0x00	Not Used	
22	0xb6		
23	0x00	0xb6	Radius = 182
24	0x00	Not Used	
25	0x00	Not Used	
26	0x12		
27	0x00	0x12	X Percent = 18
28	0x00	Not Used	
29	0x00	Not Used	
30	0xe0		
31	0xff	0xffe0	Y Percent = -32
32	0x00	Not Used	
33	0x00	Not Used	
34	0x00	Not Used	
35	0x00	Not Used	
36	0x00	Not Used	
37	0x00	Not Used	
38	0x00	Not Used	
39	0x00	Not Used	
40	0x00	Not Used	
41	0x00	Not Used	
42	0x00	Not Used	
43	0x00	Not Used	
44	0x00	Not Used	
45	0x00	Not Used	
46	0x00	Not Used	
47	0x00	Not Used	

48	0x00	Not Used
49	0x00	Not Used

Conductivity

0x90	Status byte
0x6f	Inverse Status byte
0x46	
0x83	
0x01	
0x00	0x018346, 99142. Conductivity = 99.142
0xc0	
0x00	
0x00	
0x00	0xc0, 192. Lift-Off = 0.192
0x6a	
0x9f	
0xc2	
0x00	0xc29f6a Used during calibration only.
0x19	
0x05	
0x01	
0x00	0x010519 Used during calibration only

Cscan_Encoder_Triggered

Use previous examples for exact byte format but the data is returned as 4 bytes making up each signed 32bit integer:

X1, X2, X-Mix, Y1, y2, y-Mix, Encoder2, Encoder1.

0	0xB0	Status byte
1	0x4f	Inverse Status byte
2-5		X1
6-9		X2
10-13		X Mix
14-17		Y1
18-21		Y2
22-25		Y Mix
26-29		Encoder 2 value
30-33		Encoder 1 value

Xml_Header

When connecting to the instrument to receive real time data, or changing the type of realtime data required the instrument will send up XML that contains all of the required information to understand the following real time data, ie Gains, Filters, Phases etc etc.

First the size of the XML in bytes is sent. This is sent the same as with a file transfer, see **File_Size** above.

After the size comes the Status byte for the XML Header and its inverse then all of the XML data.

Receiving a File from the Instrument

When requested to from the host PC the instrument will send a file over the VCP. This is simply a combination of the above commands.

First, a FILE_SIZE packet is sent so the host knows how many bytes to expect.

Then the entire contents of the file are transmitted using a FILE_DATA packet.

Finally a FILE_NAME packet is sent which contains the file name of what has just been transmitted.

RS232 Communications

The Victor 2.2D COM port can be configured to any of the following Baud rates in the IO menu:

9600

19200

38400

57600

The other settings of the COM port are 8 Data bits, NO parity, 1 Start and 1 Stop bit.

All communications to and from the Victor are instigated from the HOST (ie PC or PLC). Commands to perform a basic action are sent as are requests for data.

The following sections show the format of all commands.

Remote Navigation Commands

The first byte is the length of the command and includes itself. The commas (',') are NOT sent, they are just to separate the bytes for clarity in this document and in the ETHERRealtime application.

2 nd Byte	3 rd Byte	Description	Example
0x0	0x0B	OK/Power key	0x3,0x0,0xB
	0x01	Menu/Cancel key	
	0x02	BALANCE 2 (lower Balance key)	
	0x03	UP key	
	0x04	DOWN key	
	0x05	LEFT key	
	0x06	RIGHT key	
	0x07	BLANK 2 (lower Blank key)	
	0x08	BALANCE 1 (upper Balance key)	
	0x09	BLANK 1 (Upper Blank key)	
	0x0A	FREEZE key	
	0x0D	Long press on Menu/Cancel key	
	0x13	Long press on BLANK 2	
	0x15	Long press on BLANK 1	
	0x17	Long press on OK/Power key	

Requesting data from the Victor2.2D

2 nd Byte	3 rd Byte Param	Description	4 th Byte Channel	Example
0x3F	0x58	'X' requests X gain in 10ths of a dB	0,1 or 2.	Tx: 0x4,0x3f,0x58,0x1 Rx: 0x7,0x58,0x0,0x0,0x1,0xa4,0x1 Channel 2 Gain 42.0 dB
	0x59	'Y' requests Y gain in 10ths of a dB	0,1 or 2.	Tx: 0x4,0x3f,0x59,0x0 Rx: 0x7,0x59,0x0,0x0,0x1,0x90,0x0 Channel 1 Gain 40.0 dB

	0x46	'F' requests Frequency in Hz	0 or 1.	Tx: 0x4,0x3f,0x46,0x0 Rx: 0x7,0x46,0x0,0x16,0xe3,0x60,0x0 Channel 1 Freq. 1500000 (1.5MHz)
	0x50	'P' requests Phase in 100ths of a degree.	0 or 1.	Tx: 0x4,0x3f,0x50,0x1 Rx: 0x7,0x50,0x0,0x1,0xad,0xb0,0x1 Channel 2 Phase 110
	0x4C	'L' requests Low pass Filter in 100ths of a Hz.	0 or 1.	Tx: 0x4,0x3f,0x4c,0x0 Rx: 0x7,0x4c,0x0,0x0,0x75,0x30,0x0 Channel 1, 300Hz
	0x48	'H' requests High Pass Filter in 100ths of a Hz.	0 or 1	Tx: 0x4,0x3f,0x48,0x1 Rx: 0x7,0x48,0x0,0x0,0x1c,0x20,0x1 Channel 2, 72Hz
	0x78	'x' Requests live data. 16bit X and Y. These are the screen coords of the spot.	0, 1 or 2.	Tx: 0x4,0x3f,0x78,0x0 Rx: 0x7,0x78,0xff,0x1d,0xff,0xbb,0x0 Channel 1, X= -227, y= -69
	0x56	'V' requests the Firmware Version.	0	Tx: 0x4,0x3f,0x56,0x0 Rx: 0x7,0x56,0x0,0x1,0x2,0x2,0x0 Version 1.2.2

Setting data on the Victor2.2D

1st Byte (Length)	2nd Byte Param	Description	6 th Byte Channel	Example Tx. (Rx is the same message echoed back)
0x6	0x58	'X' sets X gain in 10ths of a dB	0,1 or 2.	Tx: 0x6,0x58,0x0,0x0,0x33,0x2 Channel 2 (mix) X Gain 5.1dB
	0x59	'Y' sets Y gain in 10ths of a dB	0,1 or 2.	Tx: 0x6,0x59,0x0,0x1,0xe0,0x1 Channel 2 Y Gain 48.0dB
	0x46	'F, Frequency in Hz	0 or 1.	Tx: 0x6,0x46,0x3,0xd,0x40,0x1 Channel 2 Freq. 200kHz
	0x50	'P' sets Phase in 1000ths of a degree.	0 or 1.	Tx: 0x6,0x50,0x1,0xd4,0xc0,0x0 Channel 1 Phase 120 degrees.
	0x4C	'L' sets Low pass Filter in 100ths of a Hz.	0 or 1.	Tx: 0x6,0x4c,0x0,0xc3,0xb4,0x0 Channel 1, Low Pass Filter 501Hz
	0x48	'H' sets High Pass Filter in 100ths of a Hz.	0 or 1	Tx: 0x6,0x48,0x0,0x0,0x14,0x1 Channel 2, High Pass Filter 0.2Hz

Using ETHERRealtime, RS232

ETHERRealtime from version 1.5.4 has enhanced RS232 capability to work with the Victor2.2D firmware version 1.2.2.

On opening ETHERRealtime the first tab shows the USB and the RS232 connection options. ETHERRealtime should not be used by both simultaneously. To use the RS232, select the COM port and the Baud rate. The Baud rate must be the same on the Victor IO menu. Click Connect. Now two panes will appear, the keypad and the realtime value editor, see screen shot below:

☒ Ch1
☐ Ch2
☐ ChMix

Click to request:

Firmware:

Phase deg.

Gain X dB

Gain Y dB

☒ Off
☐ Y=X
☐ Fixed

Low Pass Hz

High Pass Hz

Frequency Hz

X,Y

↗

↘

+

Reprocess

ALARM!

ALARM!

+

Blank

Freeze

↶

⏻

↷

+

Blank

📄↺

Keypad

The keypad on the right is identical to that on the Victor2.2D instrument. Clicking on a key will have the same effect as using the keypad directly on the instrument, this includes long presses of keys (ie Blank and Freeze).

Settings parameters

On the left are the parameters that can easily be configured on the instrument. PLEASE NOTE these are NOT updated to reflect the values currently on the instrument.

Clicking on the text, ie **Frequency**, will send a request to the Victor top return the frequency for the channel that is selected by the top radio buttons. The bytes transmitted and received from the Victor2.2D are shown in Hexadecimal. A translation of this in English is displayed in the central Console. All of the text

can be clicked and the instrument will return the current parameter.

To change the value of a parameter on the Victor either click on the box and enter the desired value followed by ENTER or use the up and down arrows to change the value. Each time a value changes a command is transmitted to the instrument. As before, the bytes transmitted and received are shown in Hexadecimal and an English translation displayed.